darling-lol: Running League of Legends on Linux via Darling

Oct 26, 2026 · @VINICIUS DIAS BARROS

Overview

darling-lol runs the **macOS build** of the Riot Client and League of Legends on x86-64 Linux by patching Darling, the open-source macOS compatibility layer, until it can host Riot's unmodified Mac binaries. It does not use Wine, does not touch the Windows build, and does not patch the client, game, CEF or anti-cheat.

The short answer to how the port works: the game is an x86-64 Mach-O program, so its code already runs natively on the CPU. What it lacks on Linux is everything around it: the Darwin kernel (Mach traps, BSD syscalls), Apple's userland (libSystem, Objective-C, CoreFoundation, Foundation, AppKit, Security, CFNetwork), and the graphics and audio stacks (Metal, QuartzCore, CoreAudio). Darling provides open re-implementations of those. darling-lol carries **30 per-repository patches** that close the gaps Riot's binaries hit, plus a launcher, installers, diagnostics and about 50 standalone regression probes.

Fact	Value
Upstream	lucannez64/darling-lol, 571 commits on `main` (local checkout `~/Projects/darling-lol`)
Base	Darling pinned at `60ba801d`, plus 30 pinned sub-repositories in `upstream/manifest.json`
Licence	Original tooling GPL-3.0; patches keep each target file's licence
Status	Development snapshot; manually played ARAM games run
Performance (Ryzen 5 5500 / RX 6600)	118–135 FPS average in ARAM, versus 170–190 on Windows (project's own `netstats.csv` figures, 2026-10-14)
Account risk	The author warns of permanent bans and a 3-day `UNSUPPORTED_THIRD_PARTY_TOOLS` suspension; do not play ranked or use a main account

Background: why the macOS build and Darling

The project targets the Mac client because Darling can host it without touching the game, while the Windows route is blocked. On Windows, League requires Riot Vanguard, a kernel-mode anti-cheat driver that Wine and Proton cannot load (general knowledge, not stated in the repo; the README asks Riot for Linux support "including your kernel anticheat"). The macOS client ships as an ordinary x86-64 Mach-O app bundle with no kernel driver.

Darling (darlinghq/darling) is to macOS what Wine is to Windows, with one big difference: it reuses Apple's own open-source userland (XNU, libSystem, objc4, CoreFoundation, Security) wherever Apple published it, and re-implements the closed parts (AppKit via Cocotron, Metal via Indium, CoreAudio, QuartzCore). It works in three layers:

- **mldr**, a Linux ELF loader that maps Mach-O binaries and dyld into a Linux process, sets up the Darwin commpage, and runs the code natively on the CPU.
- **libsystem_kernel emulation** (from the `xnu` repository), which turns Darwin system calls into Linux syscalls, or into RPC calls when Linux has no equivalent.
- **darlingserver**, a user-space process that runs a slice of the real XNU kernel ("duct-tape") to provide Mach ports, Mach messages, semaphores, psynch and kqueue for every guest process.

All guest processes live in a Linux mount and PID namespace rooted at the prefix (`~/.darling`), started by a guest `launchd`.

Stock Darling cannot run the Riot Client or the game: it crashes on CEF's sandbox probe, lacks APIs that Electron and the Riot Client call, has no usable Metal path for a real game, and is too slow on its RPC-heavy synchronisation path. darling-lol is the set of fixes for all of that.

Repository layout

The repository holds only patches, tooling and docs; the Darling sources, build output and the game live under the ignored `build/` directory (several GB once built) and the prefix.

Path	Contents
`upstream/manifest.json`	Darling base revision `60ba801d`, the revision of every patched submodule, its patch file, modified-file count and new files
`patches/`	31 patch files, one per repository (about 53,000 lines in total)

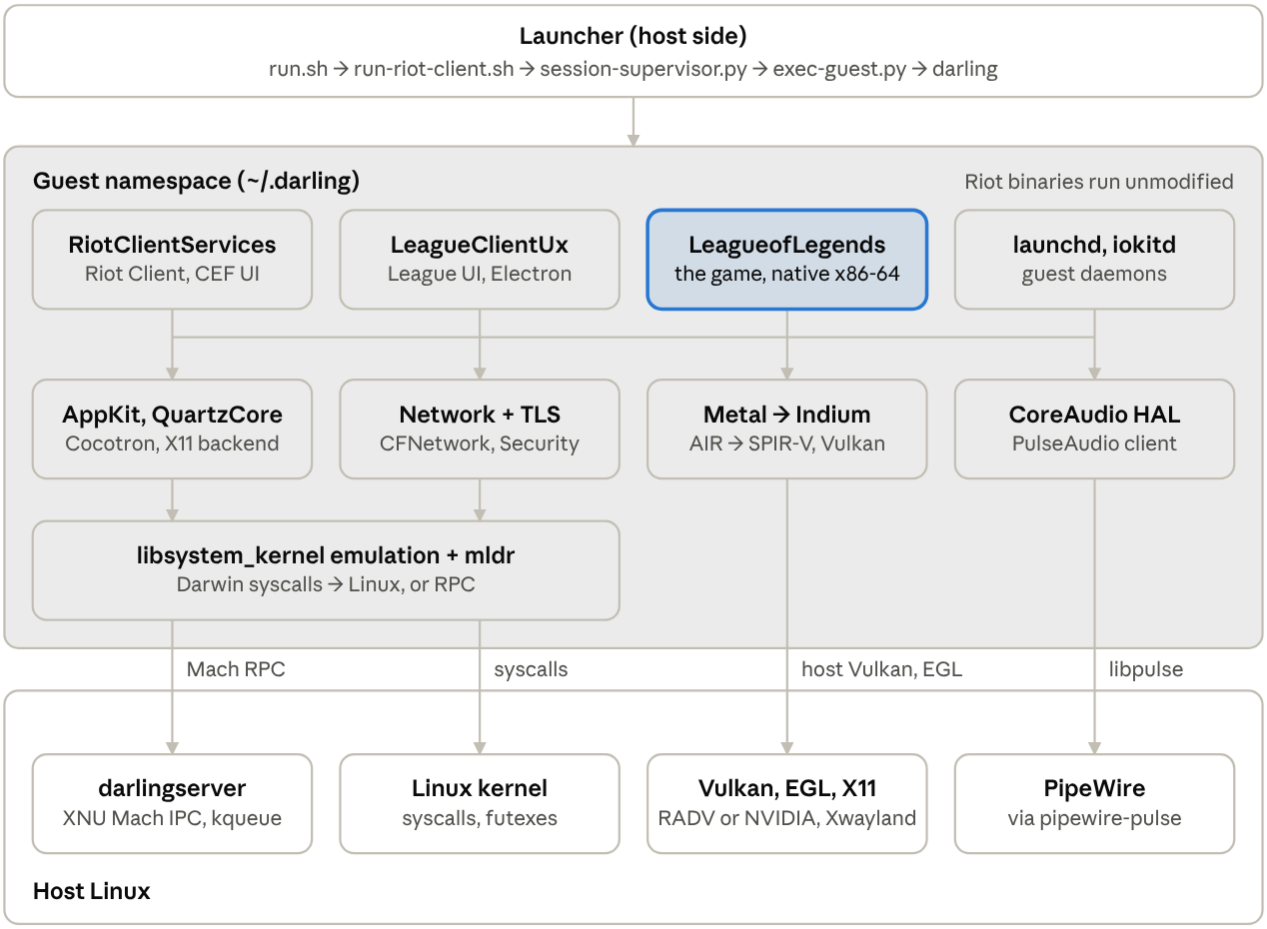
Path	Contents
`scripts/`	Entry points: `configure.py`, `prepare.py`, `preflight.py`, `build.sh`, `install-runtime.sh`, `installer.sh`, `run.sh`, `update.sh`, Nix helpers, desktop entry
`tools/`	Tokenised runtime templates, per-component builders (`build-riot-*.py`), installers (`install-riot-*.py`), the session supervisor, `exec-guest.py`, `doctor.py`, `crash-report.py`, window helpers, and about 50 standalone regression probes
`tests/`	50+ Python unit tests for the launcher, supervisor, installers, preflight, platform identity and helpers
`docs/`	`SETUP`, `STATUS`, `COMPATIBILITY`, `TESTING` (740 lines), `TROUBLESHOOTING` (951 lines), `NIX`, `PLATFORM_IDENTITY`, `INTEROPERABILITY`, `COMPLIANCE`
`flake.nix`	Experimental rootless Nix package (`#installer`, `#default`, `#shell`)
`darling-lol-launcher.sh`	Top-level launcher used by the desktop entry
`build/` (ignored)	`darling-src/` (patched source + CMake tree), `runtime/tools/` (rendered templates), `venv/`, `riot-*` artifacts, backups

`configure.py` renders `@PROJECT_ROOT@`, `@HOST_HOME@`, `@GUEST_HOME@`, `@GUEST_USER@`, `@DPREFIX@` and `@DARLING_SDK@` into `build/runtime/tools`. That is why the launcher runs `build/runtime/tools/session-supervisor.py` and not the template in `tools/`.

Architecture

Every Riot process runs as a normal Linux process inside one Darling namespace. Mach and BSD calls go to the kernel emulation library, which turns them into Linux syscalls or into RPC to darlingserver. Graphics and audio frameworks load host ELF libraries (Vulkan, EGL, libpulse) directly.

Riot's Mac binaries run unchanged; Darling supplies the Mac around them



darling-lol runtime architecture · guest namespace over host Linux

The game box is the only part that runs Riot's code; everything else in the guest is Darling plus the patches. darlingserver was the main bottleneck: in the 13:04 match the game's main thread waited 1.36 ms per frame on it, and after the game-only local mutexes, condvars and semaphores it waited 0.10 ms (16:26 match).

Porting work: what the patches change

The port is not one trick. It is hundreds of compatibility fixes, each found by running Riot's binaries, watching where they crashed, stalled or misrendered, and reproducing the failure in a standalone probe before fixing Darling. Nothing in Riot's binaries is modified. The five largest patches (Darling core, Cocotron, Indium, XNU emulation, iokitd) make up about 80% of the changed lines.

Patch	Lines	Files changed / new	What it does for League
`darling.patch` (core tree)	11,467	89 / 16	CoreAudio and AudioToolbox rewrites (PulseAudio HAL, AUHAL, AUGraph, AudioConverter shim), LaunchServices host URL hand-off, OpenGL async presenter and GL resolvers, IOSurface across processes, CoreText font enumeration, CarbonCore `Gestalt` version fix, mldr commpage CPU caps, sandbox, AVFoundation and CoreLocation stubs
`cocotron.patch` (AppKit, QuartzCore, Onyx2D)	11,833	115 / 2	X11 backend (EGL visuals, multi-monitor geometry, focus, drag and drop, decorations), `CAMetalLayer` and drawables, `CARenderer` IOSurface upload, `NSView setFrame:` dispatch that Chromium needs for resize, event polling, open and save panels
`metal--deps--indium.patch` (Indium + Iridium)	8,969	36 / 14	Metal-on-Vulkan engine: AIR to SPIR-V translation (LLVM 22/23), submit batching, a 32 MB memory pool, render-pass caching, retirement queue, deferred encoders, null descriptors, depth/stencil handling, GPU timing diagnostics
`xnu.patch` (libsystem_kernel emulation)	6,296	65 / 10	`mac_syscall` probe reported as satisfied for CEF, process-local mutexes, condvars and Mach semaphores in the game, real CPU topology in `sysctl`, `MADV_FREE` mapping, fast clock paths, `change_fdguard_np`, `PT_DENY_ATTACH`
`metal.patch` (Metal wrapper)	3,882	27 / 12	Objective-C Metal API on top of Indium: argument reflection, blit encoder, bind deduplication, vertex-descriptor dirty masks, device naming (`AMD Radeon Pro 5500M`) and VRAM budget
`iokitd.patch`	3,636	17 / 12	`IOPlatformExpertDevice` with platform identity, power management (`IOPMrootDomain` , logind sleep, X11 power assertions), `IOSurfaceRoot` , display checks without crash loops
`darlingserver.patch`	2,957	36 / 1	Reusable receive buffers, fileports, kqueue channels, timers, `RLIMIT_NOFILE` raise for memfd-backed memory entries, `MEMDIAG` logging

Patch	Lines	Files changed / new	What it does for League
`configd.patch`	744	2 / 0	`SystemConfiguration` stubs and DNS info that follows the host resolver
`libplatform.patch`	636	6 / 1	AVX2 string primitives (`memchr` , `memcmp` , `memmove` , `bzero` , `memset_pattern`)
`libkqueue.patch`	617	13 / 0	kqueue lifetime, timer and Mach-port filter fixes used by libdispatch
`corecrypto.patch`	494	4 / 0	ECDSA and modular arithmetic fixes for TLS and code-signing checks
`corefoundation.patch`	323	5 / 0	Run-loop and CF fixes
`cfnetwork.patch`	305	4 / 0	Missing `NSURLSession` methods (`dataTaskWithRequest:completionHandler:` , `finishTasksAndInvalidate`) and `CFHost` cancel crash
`security.patch`	240	9 / 0	Code-signing parser and `trustd` network fixes
`libxpc.patch`	182	3 / 0	XPC connection, event and pipe ownership
16 small patches	under 130 each	1–2 each	Foundation, Libinfo (IPv4-only DNS for `riotcdn.net`), libmalloc hints, libresolv, libnotify, libtrace, libpthread, libdispatch, libcxxabi, objc4 (debug macros), OpenDirectory, IOKitUser, curl, libnetwork, compiler-rt, cctools-port

The fixes fall into six groups:

1. **Getting CEF and Electron to start.** Chromium probes `___mac_syscall` for the sandbox; Darling returned `ENOSYS` , which CEF treats as fatal, so `mac_syscall.c` now reports the probe as satisfied (the guest is still unsandboxed). `MAC_CHROMIUM_TMPDIR=/private/tmp` fixes the "Not Found" UI caused by failed `app.asar` copy-outs. CoreText gained `CTFontDescriptorCreateMatchingFontDescriptors` and fontconfig enumeration so the language dropdown stops crashing the renderer.
2. **Missing or wrong APIs the Riot Client calls.** `NSURLSession` completion-handler tasks, `CFHostCancelInfoResolution` (a `CFRetain(NULL)` trap), six LaunchServices null error-pointer dereferences, and `Gestalt` reporting macOS 10.10 instead of 14.7.
3. **Platform identity.** iokitd exposes a per-prefix synthetic serial, UUID, MLB and ROM through `IOPlatformExpertDevice` , generated on first install by `configure-darling-platform.py --provision` . `system_profiler` and `MTLDevice.name` report a consistent `MacBookPro16,1` with an `AMD Radeon Pro 5500M` .

4. **Rendering the game.** Metal to Vulkan through Indium, AIR shaders to SPIR-V through Iridium, and presentation through EGL on X11 (next sections).
5. **Networking.** DNS follows the host's `/etc/resolv.conf`, the Riot CDN resolves IPv4 only (its downloader lacks IPv6 `CONNECT_TO` support), and a root-certificate store is ensured at every launch.
6. **Performance.** Moving hot synchronisation out of darlingserver, optimised libc and libc++ overlays, real CPU topology, and Indium submit batching took ARAM from 84 to 135 FPS (see Known issues).

Graphics, audio, input and networking

Graphics: Metal → Vulkan → EGL → X11

The Mac build of League renders with Metal, which Linux does not have, so every frame crosses four translation layers before it reaches the screen.

1. **Metal API** (`metal.patch`): the game calls the Objective-C Metal classes (`MTLDevice`, `MTLRenderCommandEncoder`, `MTLBlitCommandEncoder`). The wrapper forwards to Indium and drops redundant work, such as re-binding the same texture, sampler, buffer or pipeline at the same slot. `MTLDevice.name` reports an `AMD Radeon Pro 5500M` for AMD or NVIDIA hosts, with VRAM from `DARLING_METAL_VRAM_MB` (default 4096).
2. **Shaders: AIR → SPIR-V** (Iridium, inside Indium): Metal shader libraries contain AIR, Apple's LLVM bitcode. Iridium loads the host's LLVM (22 or 23) and translates AIR to SPIR-V for Vulkan. The patch handles LLVM 23's split branch opcodes and vector-splat constants, which otherwise made every game shader fail and crashed the game before the loading screen. Mesa's shader cache is redirected to `~/.cache/darling-lol`, because the guest `$HOME` (`/Users/<user>`) does not exist on the host and the cache was disabled.
3. **Indium, Metal on Vulkan**: command buffers become Vulkan command buffers. League commits about 30 small command buffers per frame, so Indium merges them into one `vkQueueSubmit2` batch (500 µs window by default). Resources share 32 MB memory blocks instead of one `VkDeviceMemory` each. Render passes are cached by exact description, null descriptors stand in for unbound resources, and completed work retires on a queue.
4. **Presentation** (QuartzCore `CAMetalLayer`): the drawable's Vulkan memory and semaphores are exported as `OPAQUE_FD` and imported into the layer's OpenGL (EGL) context, which draws into the X11 window. This only works when Vulkan and OpenGL

run on the same GPU, so the device UUIDs are compared and logged to the guest `/tmp/metal-device.log`. The window itself comes from Cocotron's X11 backend (AppKit), which runs under Xwayland on Hyprland.

The Riot Client and League client UIs are Chromium (CEF and Electron). They render through OpenGL into IOSurfaces. Darling has no `IOSurfaceRoot` kernel service, so IOSurfaces are user-space objects whose backing file descriptor travels between processes as a Mach fileport. `CARenderer` uploads them as textures.

Runtime tuning uses guest flag files such as `/tmp/metal-submit-batch-us` and `/tmp/metal-no-lazy-wakeup`, listed in `docs/SETUP.md`.

Audio: CoreAudio → PulseAudio API → PipeWire

Darling's CoreAudio HAL is a PulseAudio client, so on PipeWire systems it goes through `pipewire-pulse`. Each stream appears in `wpctl status` as **"Darling CoreAudio"** (`application.process.binary = mldr`). The patch rewrote most of it:

- **Output pacing.** Real CoreAudio calls an output IOProc once per I/O cycle, evenly spaced, and Chromium's `AUHALStream` depends on that. PulseAudio's write requests are bursty, so a dispatch timer now ticks at the block period and renders exactly one block per tick, with bounded catch-up after a late wake-up.
- **Drift servo.** The sink clock is not the guest clock. If the buffer stays under-filled for a whole window, the stream runs about 6% faster for the next window.
- **Input.** Capture fragments are one device buffer long for voice latency, with a bounded server backlog. Holes are delivered as silence, and one failed IOProc no longer mutes the microphone for the session.
- **Lifetime.** An IOProc may stop its own stream; callbacks are serialised on a PulseAudio dispatch queue to avoid deadlocks and use-after-free.
- **New pieces.** `AUGraph`, an AUHAL and default-output audio unit, a matrix mixer, and an `AudioConverter` shim that CoreAudio re-exports (its comment names Discord voice as a user).

Input

Cocotron's X11 backend turns X events into `NSEvent`s. The patches fix multi-monitor coordinates (macOS's global origin is the XRandR primary's top-left), drag hit-testing through Electron's control-region overlays, relative mouse under Xwayland, cursor shapes, event coalescing and scroll events. A `SecureInput` stub and text-input-source fixes cover HIToolbox. `XMODIFIERS=@im=none` disables the host input method inside the guest.

Networking and system services

- **DNS:** the guest `/etc/resolv.conf` links to the host's through `/Volumes/SystemRoot`, instead of Darling's built-in Google DNS.
- **Riot CDN:** Libinfo suppresses AAAA lookups for `*.riotcdn.net` only (`RCLOL_CDN_IPV4_ONLY=1`).
- **TLS:** `ensure-darling-root-certificates.py` builds the guest root store at launch; corecrypto and `trustd` fixes let certificate checks pass.
- **Links:** LaunchServices opens `http`, `https` and `mailto` links in the host browser through `xdg-desktop-portal` or `xdg-open`.
- **Power:** iokitd maps macOS power assertions to host inhibitors and logind sleep and wake to `IOPMrootDomain` notifications.

Launch flow: from `run.sh` to in-game

`./scripts/run.sh` takes twelve steps to reach a running Riot Client. The game is then started by the client itself, inside the same Darling namespace.

1. **`scripts/run.sh`** checks Nix readiness (rootless installs only), puts `/usr/local/bin` and the venv on `PATH`, and re-renders the templates with `configure.py`. It then runs `build/runtime/tools/run-riot-client.sh`, wrapped in `gamemoderun` when that is installed (`RCLOL_GAMEMODE=0` disables it).
2. **Opt-in recovery**: `RCLOL_RECOVER_STALE_CLIENT`, `RCLOL_RECOVER_HIDDEN_CLIENT` and `RCLOL_CLEAN_STALE_HELPERS` clean up a client left without a window or orphaned helpers.
3. **Runtime probe**: `session-supervisor.py --probe-runtime` records whether `darlingserver` was already running. That decides later whether this launch owns cleanup.
4. **Prefix repair**: `ensure-darling-runtime.py`, `ensure-darling-service-accounts.py`, `ensure-league-runtime-layout.py` and `ensure-darling-root-certificates.py` check the prefix, guest accounts, the League app layout and the TLS root store.
5. **Client location**: if `/Applications/Riot Client.app` is missing, it is symlinked to `/Users/Shared/Riot Games/Riot Client.app`, where Riot's installer puts it.
6. **Optional match helpers**: with `RCLOL_CLIENT_IN_GAME_TRICKS=1`, the launcher reads `LCULocalPreferences.yaml` and starts either the close helper or the hide helper for the League client during matches.
7. **Host environment**: it sets `MESA_SHADER_CACHE_DIR`, `MESA_NO_ERROR=1`, NVIDIA's `__GL_MaxFramesAllowed=1` and `__GL_YIELD=NOTHING`, reads the monitor refresh rate from `xrandr` into `RCLOL_REFRESH_HZ`, and sets timer slack to 1 ns.

8. **`session-supervisor.py`** starts the guest as a child. It forwards SIGINT, SIGTERM and SIGHUP. When the client exits it asks darlingserver to shut down and stops anything left in the guest PID namespace, identified by the `__mldr_sockpath` variable. It only cleans up a runtime it started itself.
9. **`exec-guest.py`** closes every inherited file descriptor above stderr. With `RCLOL_FILTER_HOST_ENV=1` it also strips host desktop variables such as `WAYLAND_DISPLAY`, `GTK_*` and `SSH_*`. It always keeps `DISPLAY`, PulseAudio/PipeWire, Mesa, `__GL_*` and `VK_*`.
10. **`env … darling RiotClientServices`**: the guest environment unsets `DYLD_INSERT_LIBRARIES` and `ELECTRON_RUN_AS_NODE`, and sets `EGL_PLATFORM=x11`, `MAC_CHROMIUM_TMPDIR=/private/tmp` and `XMODIFIERS=@im=none`. It points `DYLD_LIBRARY_PATH` at `/Users/<user>/RuntimeLibraries` and `DYLD_FRAMEWORK_PATH` at `/Users/<user>/RuntimeFrameworks`, so the optimised overlays shadow the SDK copies. Arguments: `--region=EUW --locale=en_GB --app-root=/Applications/Riot Client.app --launch-product=league_of_legends --launch-patchline=live --log-dir=…`.
11. **`darling`** creates the namespace if needed. It starts `darlingserver` and the guest `/sbin/launchd`; launchd starts guest daemons such as iokitd and configd. Then `mldr` loads `RiotClientServices` with dyld.
12. **Riot Client → League client → game**: RiotClientServices launches `LeagueClient` and its Electron UI (`LeagueClientUx`). Clicking Play starts `LeagueofLegends.app/Contents/MacOS/LeagueofLegends`. The kernel library recognises that path and turns on the game-only fast paths (local mutexes, condvars, semaphores), logging its choices to `/tmp/rclol-game-decisions.log`.

Build and install

A native install takes seven commands and at least 50 GB of disk. Run everything as your normal user except the two `sudo` steps.

```
git clone https://github.com/lucannez64/darling-lol.git && cd darling-lol
python3 -m venv build/venv && build/venv/bin/pip install -r requirements.txt
python3 scripts/configure.py --guest-user "$USER"
./scripts/build.sh
sudo cmake --install build/darling-src/build
./scripts/install-runtime.sh
./scripts/installer.sh euw   # first time: sign in and install League
./scripts/run.sh             # every later launch
```

What each step does:

Step	Effect
`preflight.py`	Stops early unless Python 3.10+, CMake 3.20+, Git LFS 2.13+, Linux 5.14+, LLVM 16+ (22/23 tested) and Vulkan 1.3 with timeline semaphores on one device are present. `RCLOL_SKIP_PREFLIGHT=1` bypasses it
`configure.py`	Renders paths into `build/runtime/tools` and saves `build/config.json`
`prepare.py`	Clones Darling at `60ba801d`, checks every submodule revision against the manifest, and applies the patches. It never resets a checkout. It redirects `darling-swift` Git LFS from the retired `git-lfs.darlinghq.org` to GitHub
`build.sh`	CMake with Unix Makefiles, empty build type (objc4 debug-macro check), `TARGET_i386=OFF`, `ENABLE_METAL=ON`, `-O3 -march=native -mtune=generic`. `bsdthread_register.c` and `sigexc.c` stay at `-O0` because they depend on stack layout
`cmake --install`	Base SDK in `/usr/local/libexec/darling`, `darling` in `/usr/local/bin`
`install-runtime.sh`	Creates `~/.darling`, `private/tmp` (mode 1777) and the root store. On a new prefix it provisions a platform identity. Installs the presentation and GPU overlays (AppKit, QuartzCore, OpenGL, Metal, Indium, CoreText), the `-O3` libc, libc++, libc++abi and libsystem_platform overlays, and `system_profiler`, then runs `validate-riot-runtime.py` and the thread-local miscompile check
`update.sh`	After `git pull`: reverts and re-applies changed patches, rebuilds incrementally and reinstalls. Refuses to run while Riot or League is open

Two compiler traps the project documents:

- `-mtune=native` or `-mtune=znver*` with Clang 23 passes the Darwin thread-local descriptor in the wrong register. Every `__thread` access returns NULL and launchd crashes at boot. Keep `-mtune=generic`.
- `libsystem_m` must stay at `-O0`: an `-O2` rebuild turned the Riot Client window black.

Activation: installers replace files atomically but never restart anything, and Darling's overlay mount can keep serving old libraries. After any update, exit the game and both clients, then run `darling shutdown` and `sh scripts/run.sh`.

Nix alternative: `nix run github:lucannez64/darling-lol#installer` builds into `~/.local/share/darling-lol` rootless, using `unshare` and bubblewrap, with one workspace session at a time.

Known issues and limitations

The biggest open risks are account bans and a frame-rate gap with Windows; the rest are specific compatibility gaps.

Issue	Status
Account risk	The author reports 3-day `UNSUPPORTED_THIRD_PARTY_TOOLS` suspensions and expects permanent bans. Use a throwaway account, no ranked
FPS gap	ARAM 118–135 FPS average versus 170–190 on Windows (Ryzen 5 5500 / RX 6600). The game's main thread spends about 72% of its time in its own code, limited by memory latency. Every Metal call still crosses Objective-C, the wrapper, Indium and Vulkan
VAN -101	Some launches report an unsupported system configuration. Successful sessions logged `MvgState: 0`, failed ones `MvgState: 5`. No verified fix
Idle install	Native LaunchAgent registration can return HTTP 409 and leave the download idle. No runtime-only fix
Practice Tool exit	One report of the game exiting about 2 seconds after start, cause unknown
`NSURLSession`	Partial: cancelling a task does not interrupt a transfer already in progress
Security	The guest is unsandboxed: the CEF sandbox probe is answered as satisfied but nothing enforces it

Issue	Status
RAM	About 4.3 GiB PSS for clients and runtime. The `MADV_FREE` fix works in probes; its real-client effect is unmeasured
Portability	A fresh checkout, build and install has not been validated end to end on another machine. Custom prefixes are untested

Troubleshooting and debugging

Start with the host-side doctor: it starts nothing and works while a client is open.

```
darling-lol --doctor                          # GPU, Vulkan, display, memory,
disk, prefix
darling-lol --doctor --report doctor.json   # machine-readable
darling-lol --report                          # crash bundle in
~/.cache/darling-lol/reports/
darling-lol --perf                            # frame pacing from the game's
own logs
```

(`darling-lol` is the Nix launcher; on a native install run `python3 tools/doctor.py` and `python3 tools/crash-report.py`.)

Symptom	Where to look / what to do
An update seems to have no effect	Exit everything, `darling shutdown`, `sh scripts/run.sh`. The overlay mount keeps old libraries until the namespace restarts
Launcher stays attached after closing the window	Closing the window does not quit the client. Use tray → Quit, or `RCLOL_STOP_AFTER_CLIENT_SECONDS=120`
Black game window, especially NVIDIA	Guest `/tmp/metal-device.log` (enable with `DARLING_METAL_DEVICE_LOG=1`): Vulkan and GL must be on the same GPU
Shaders recompiled every start	`MESA_SHADER_CACHE_DIR` must be a writable host path (default `~/.cache/darling-lol`)
"NO INTERNET FOUND"	`darling shell curl --noproxy '*' https://clientconfig.rpg.riotgames.com/`; check the prefix `private/etc/resolv.conf`

Symptom	Where to look / what to do
UI crashes after Play (`resource shortage (6)`)	`Max open files` in `/proc/$(pgrep -x darlingserver)/limits` ; look for `MEMDIAG` in `dserver.log`
`iokitd` restart loop, `LOLSIGFAIL addr=0x143`	Start Darling with `EGL_PLATFORM=x11` and a readable `XAUTHORITY`
Low FPS	Check `/tmp/rclol-game-decisions.log` shows `local_semaphores=1 core_isolation=0` . On NVIDIA try `/tmp/metal-submit-batch-us` = `5000` and `/tmp/metal-submit-batch-max` = `64`
Cursor shape frozen in fullscreen (Hyprland)	`render:direct_scanout = 0`
Click offset or input issues	`darling shell touch /tmp/rclol-input-trace` , then read `/tmp/rclol-input-trace.log`
Resize clipping	`DARLING_UI_GEOMETRY_TRACE=1 sh scripts/run.sh` ; read `~/.darling/private/tmp/darling-ui-geometry.log`

Guest flag files (`/tmp/metal-*` , `/tmp/rclol-*`) must be created from inside the guest, for example `darling shell touch /tmp/metal-debug-waits` . A file created on the host in `~/.darling/private/tmp` while Darling runs may stay invisible to running guest processes.

Regression probes: each fix has a standalone probe in `tools/` (for example `riot-cfhost-cancel-check.c` , `riot-urlsession-check.m` , `riot-audio-unit-check.cpp`) that reproduces the old failure without launching Riot. `docs/TESTING.md` lists how to build and run them; `python3 -m unittest discover tests` runs the host-side unit tests.

Project rules: do not attach a debugger to, inject into, query the APIs of, or automate input for the client or game. Keep logs, tokens and account data private; the crash bundle redacts home, user and host names unless `--no-redact` is passed.